

Terraform, from First Principles

The practical fundamentals of infrastructure as code

Marcel Barlik

18 August 2026

Today: 30 Minutes to a Safe First Workflow

- ▶ **What Terraform is** (5 min) — infrastructure as reviewed, repeatable code
- ▶ **Define the contracts** (8 min) — providers, resources, inputs and locals
- ▶ **Use the workflow** (8 min) — init, fmt, validate, plan and apply
- ▶ **Work safely as a team** (5 min) — state, credentials and review

Outcome

You will be able to read a Terraform configuration, make a small infrastructure change, review its plan, and know when *not* to apply it.

What Is Terraform?

Definition

Terraform is an **Infrastructure as Code (IaC)** tool. The infrastructure and configuration defined in files is the source of truth. Terraform compares that desired state with the real world, and based on this it proposes the necessary changes.

It manages

- ▶ Cloud resources: networks, compute, storage, databases
- ▶ SaaS configuration: dashboards, teams, access controls
- ▶ Internal platforms exposed through APIs

It gives you

- ▶ A version-controlled description of intent
- ▶ A preview of changes before they happen
- ▶ Repeatable environments and a change history

Manual Configuration vs. Terraform

Clicking in a UI

- ▶ Easy to make an unrecorded change
- ▶ Hard to reproduce in another environment
- ▶ Review happens after the fact
- ▶ Restoring a previous layout is manual

Terraform

- ▶ Desired infrastructure lives in Git
- ▶ One configuration can target each environment
- ▶ `plan` shows the proposed change first
- ▶ History, review and rollback use normal engineering practice

Terraform does not replace cloud, SaaS, or internal platform. It makes the desired configuration explicit and repeatable.

Terraform vs. a Script

Imperative: describe the steps

Intent in a script

“Create a bucket; add these tags; then create a policy.”

- ▶ The author manages ordering and retries
- ▶ Re-running safely is extra work
- ▶ The script does not inherently know what already exists

Declarative: describe the outcome

Intent in Terraform

“This bucket, its tags, and its policy should exist.”

- ▶ Terraform builds the dependency graph
- ▶ `plan` calculates the delta from current state
- ▶ `apply` changes only what is needed

Terraform's Change Flow

Configure → Plan → Apply

Configure

- ▶ Describe the desired outcome
- ▶ Store it in version control

Plan

- ▶ Preview the proposed changes
- ▶ Review before anything changes

Apply

- ▶ Make the approved changes
- ▶ Verify the resulting infrastructure

Important boundary

plan is read-only. apply is the explicit step that changes remote infrastructure.

Terraform's Mental Model

Terraform calculates the plan by bringing together desired configuration, its memory of managed objects, and the platform it is managing.

Configuration

- ▶ .tf files
- ▶ Desired infrastructure
- ▶ Inputs and rules

State

- ▶ Terraform's record
- ▶ Maps code to remote objects
- ▶ Enables diffing

Provider APIs

- ▶ Cloud, SaaS, or internal APIs
- ▶ Reads actual state
- ▶ Creates or updates objects

The key idea

Terraform converges the remote system toward the configuration. It is not a script that blindly re-runs every line.

Start with Four Building Blocks

Most Terraform configuration is easier to read once four words are clear.

Provider

The plugin that speaks to a platform's API.

Resource

A remote object Terraform is responsible for.

Variable

A value supplied from outside the module.

Local value

A value calculated and named inside the module.

We will build one small configuration using each block in that order.

A Provider Is Terraform's API Adapter

Terraform Core understands configuration, state and the plan. A **provider** understands a particular remote system.

Terraform Core $\longleftrightarrow$ AWS provider $\longleftrightarrow$ AWS APIs

The provider defines:

- ▶ which **resource** types Terraform can manage;
- ▶ which settings those **resources** accept; and
- ▶ how to read, create, update and delete them through the API.

Examples

AWS, Azure, GitHub, Datadog and many internal platforms can all have providers.

A Provider Has Two Separate Contracts

1. Require the plugin

```
terraform {  
  required_providers {  
    aws = {  
      source  = "hashicorp/aws"  
      version = "> 5.0"  
    }  
  }  
}
```

Which provider may Terraform install?

Keep identity outside the code

Region and other non-secret settings may be configuration. Credentials should come from the platform's normal environment, profile or workload identity mechanism.

2. Configure the connection

```
provider "aws" {  
  region = var.aws_region  
}
```

How should this module use that provider?

What Happens When You Run `terraform init`?

`init` prepares the current module; it does not create infrastructure.

1. Terraform reads the `required_providers` block.
2. It downloads a compatible provider plugin.
3. It records the exact selected version in `.terraform.lock.hcl`.

Team habit

Commit the lock file. Everyone and CI then start from the same provider selection.

So far: Terraform can talk to AWS, but we have not asked it to manage anything.

A Resource Is an Object Terraform Manages

```
resource "aws_s3_bucket" "assets" {  
  bucket = "payments-staging-assets"  
}
```

- ▶ `resource` declares an object Terraform will manage.
- ▶ `aws_s3_bucket` is the resource type, defined by the provider.
- ▶ `assets` is the local name, chosen by us.
- ▶ Together, the type and name form the resource's address:
`aws_s3_bucket.assets`.

Inside a Block: Arguments and Nested Blocks

Argument

```
bucket = "payments-assets"
```

Assigns an expression to a named setting.

Nested block

```
rule {  
  status = "Enabled"  
}
```

Groups related settings into a structure.

The provider documentation is the contract

It tells us which arguments are required, which blocks are valid, and which attributes become available after creation.

References Connect Managed Objects

```
resource "aws_s3_bucket_lifecycle_configuration" "assets" {  
  bucket = aws_s3_bucket.assets.id  
  
  rule {  
    id      = "archive-old-assets"  
    status = "Enabled"  
  }  
}
```

`aws_s3_bucket.assets.id` means:

- ▶ find the resource at `aws_s3_bucket.assets`;
- ▶ read its resulting `id`;
- ▶ use that value here.

Terraform also learns the order: create or read the bucket before configuring its lifecycle.

A Variable Is an Input to the Module

The declaration defines what the module accepts:

```
variable "environment" {  
    description = "Environment this bucket belongs to"  
    type        = string  
    nullable    = false  
}
```

The resource reads that input as `var.environment`:

```
bucket = "payments-${var.environment}-assets"
```

Why declare a type?

Terraform can reject an invalid input before it attempts a remote change.

Declaring a Variable Is Not Assigning Its Value

The module declares

```
variable "environment" {  
  type = string  
}
```

The caller supplies

```
environment = "staging"
```

For example, in `staging.tfvars`.

Usually stored in `variables.tf`.

A team can instead inject values through CI or `TF_VAR_...`

Security note

Never put secrets in a committed `.tfvars` file.

A Local Names a Value Calculated Here

```
locals {  
  resource_name = "payments-${var.environment}"  
  common_tags = {  
    Environment = var.environment  
    ManagedBy   = "terraform"  
  }  
}
```

Variable: `var.environment`

Supplied by the caller; part of the module's interface.

Local: `local.resource_name`

Derived inside the module; removes repetition and gives an expression a useful name.

Now Read the Whole Value Flow

```
locals {  
  resource_name = "payments-${var.environment}"  
}  
resource "aws_s3_bucket" "assets" {  
  bucket = "${local.resource_name}-assets"  
}
```

1. Input variable

`var.environment`
Supplied as `staging`.

2. Calculated local

`local.resource_name`
Becomes
`payments-staging`.

3. Resource argument

`bucket`
Receives
`payments-staging-assets`.

The idea: one input can feed a local, and that local can then feed many resource arguments.

Files Are for Humans; the Directory Is the Module

Terraform loads all `.tf` files in one directory together as a **module**.

File	Typical purpose
<code>versions.tf</code>	Provider requirements
<code>providers.tf</code>	Provider configuration
<code>variables.tf</code>	Input contracts
<code>main.tf</code>	Resources and local values
<code>outputs.tf</code>	Values exposed to callers

Important: file order does not control creation order. References between values and resources do.

Terraform Commands: The Two Halves

```
terraform init

terraform fmt
terraform validate

terraform plan
terraform apply
```

Prepare and check

Work on the local configuration.

Preview and change

Interact with the target platform.

Run these commands from the module directory. `init` is also needed when provider requirements or the state backend change.

fmt and validate Answer Different Questions

```
terraform fmt
```

“Is the configuration formatted consistently?”

It rewrites files into Terraform’s standard style.

```
terraform validate
```

“Is the configuration internally valid?”

It checks syntax, types and references without proposing a remote change.

Both are fast feedback. Neither proves that the change is safe for a particular account or environment; that is the plan’s job.

A Plan Is Terraform's Proposed Change

terraform plan brings together three views:

desired configuration + Terraform state + current provider API data

The result is a preview of what *would* change. Planning may read remote systems and require credentials, but it does not apply the proposed infrastructure changes.

A successful plan is not approval

“Terraform can do this” is different from “we intend to do this.” Read the result.

Read the Plan as a Code Review

Focus first on Terraform's action symbols:

Symbol	Action	Question to ask
+	create	Is this object expected and named correctly?
-	destroy	Why is it being removed? Is this expected?
~	update	Is each changed field intentional?
-/+	replace	What is being replaced? Is potential down-time/data loss acceptable?

Stop condition

Do not apply an unexplained destroy or replacement. The plan is the last inexpensive place to catch a wrong value, wrong target account, or typo.

Apply Only the Plan You Understand

A safe review answers four questions:

1. Am I targeting the intended account and environment?
2. Are the resource addresses the ones I expected?
3. Can I explain every create, update, replacement and destroy?
4. Is the operational impact acceptable?

The boundary

`apply` is the explicit command that asks the provider to change remote objects.

State Is Terraform's Memory

State records which remote object belongs to each resource address.

Configuration says

- ▶ “Manage a bucket at this address.”
- ▶ “These settings are desired.”

State remembers

- ▶ “That address maps to bucket ID 123.”
- ▶ Provider metadata needed to manage it.

Do not delete state to “start over”

Removing Terraform's mapping does not remove the remote object; it makes Terraform forget that the object is already managed.

Drift Is a Difference Terraform Discovers

configuration says A $\neq$ provider API reports B

Drift often appears when someone changes a managed object directly in its UI or API.

1. A plan reads the remote object and reveals the difference.
2. The team decides which version is intended.
3. Code and infrastructure are reconciled through a reviewed change.

Habit: use Terraform as the normal route for changing objects Terraform manages.

Shared Work Needs Shared State

Local `terraform.tfstate` is suitable for learning, not for a team managing the same infrastructure.

Remote backend

Stores state in a shared, durable location so engineers and CI use the same memory.

Locking

Prevents two writers from changing the same state at the same time.

Treat state as sensitive operational data

It can contain resource details and sometimes secret values. Encrypt it, restrict access, keep backups, and never commit it to Git.

A Safe Team Change Has One Clear Path

branch → check → plan → review → apply → verify

- ▶ Keep the code change focused; separate resource changes from provider upgrades.
- ▶ Review the plan for the intended environment in the pull request.
- ▶ Let CI or a designated operator apply with environment-scoped credentials.
- ▶ Verify the remote result after apply.

The invariant

The code reviewed, the values planned and the infrastructure applied must describe the same change.

Three More Terms You Will Meet

Output A value a module exposes to its caller, such as a bucket ID.

Data source A read-only lookup of an object Terraform does not own here.

Child module A reusable Terraform package called by another module.

A sensible order for learning

Start with resources, variables and local values in one module. Add outputs when callers need results, data sources when ownership is elsewhere, and child modules when reuse is useful – just like when you program and feel the need to refactor.

Pause When the Plan Surprises You

Do not apply yet if you see:

- ▶ an unexplained destroy or replacement;
- ▶ resources from the wrong account or environment;
- ▶ unrelated changes you did not intend; or
- ▶ a provider or lock-file upgrade mixed into an ordinary resource change.

Check the selected values, credentials, provider configuration, resource references and state before trying again.

Recovery tools are not everyday shortcuts

Commands such as `-target`, `import` and direct state manipulation require a specific, understood recovery plan.

What You Should Now Be Able to Do

1. Explain the job of a provider and the difference between requiring and configuring it.
2. Read a resource, its arguments, nested blocks, variables, locals and references.
3. Follow `init` → `fmt` → `validate` → `plan` → `apply`.
4. Treat the plan as a change review and state as sensitive shared operational data.
5. Recognise when a proposed change is safe to apply—and when to stop.

One principle to remember

Never apply a Terraform plan you cannot explain.

Questions?

Suggested next step: make one safe, non-production Terraform change.

Start with any small module; plan before you apply.

References I

- ▶ Terraform documentation:
`https://developer.hashicorp.com/terraform/docs`
- ▶ Terraform CLI reference:
`https://developer.hashicorp.com/terraform/cli`
- ▶ Terraform Registry: `https://registry.terraform.io/`